

TD Pratique – Migration de Données

Entraînement guidé : 3 types de migration pas à pas

Module : ASRBD – Tests d'intégration & Migration de données

Étudiant : Amine Ech Chalyouat

Formation : Bachelor SYSOPS

Date : 19 mai 2026

Environnement : Windows 11 / WSL2 Ubuntu / Docker Desktop

Vérification complète : 2 containers Up, 4 tables créées, 5 utilisateurs, 5 fichiers présents.

2. Phase d'analyse obligatoire

A1 – Inventaire des tables

Table	Nb lignes	Dépend de	Critique ?
utilisateurs	5	—	Oui – table racine
formations	4	—	Oui – table racine
progressions	6	utilisateurs + formations	Oui – FK doubles
resultats_examens	3	utilisateurs + formations	Oui – FK doubles

```
amine@OrdiDeAmine:~/td_migration$ echo 'Support Excel avance - Module 1' > data_montpellier/supports/excel_module1.pdf
echo 'Support Cybersecurite - Introduction' > data_montpellier/supports/cyber_intro.pdf
echo 'Support gestion de projet - Methodes' > data_montpellier/supports/gestion_methodes.pdf
echo 'Video Linux debutants - Partie 1' > data_montpellier/supports/linux_video1.mp4
echo 'Image Logo TechCorp' > data_montpellier/supports/Logo.png
ls -lh data_montpellier/supports/
total 20K
-rw-r--r-- 1 amine amine 37 May 19 11:19 cyber_intro.pdf
-rw-r--r-- 1 amine amine 32 May 19 11:19 excel_module1.pdf
-rw-r--r-- 1 amine amine 37 May 19 11:19 gestion_methodes.pdf
-rw-r--r-- 1 amine amine 33 May 19 11:19 linux_video1.mp4
-rw-r--r-- 1 amine amine 20 May 19 11:19 Logo.png
amine@OrdiDeAmine:~/td_migration$ docker exec -it pg_montpellier psql -U admin -d techcorp_db -c "
SELECT 'utilisateurs' AS table_name, COUNT(*) FROM utilisateurs
UNION ALL SELECT 'formations', COUNT(*) FROM formations
UNION ALL SELECT 'progressions', COUNT(*) FROM progressions
UNION ALL SELECT 'resultats_examens', COUNT(*) FROM resultats_examens;"
 table_name | count
-----
utilisateurs | 5
formations | 4
progressions | 6
resultats_examens | 3
(4 rows)
amine@OrdiDeAmine:~/td_migration$ docker exec -it pg_montpellier psql -U admin -d techcorp_db -c "
SELECT pg_size_pretty(pg_database_size('techcorp_db')) AS taille_base;"
taille_base
-----
7710 kb
(1 row)
amine@OrdiDeAmine:~/td_migration$ docker exec -it pg_montpellier psql -U admin -d techcorp_db -c "
SELECT tc.table_name, kcu.column_name, ccu.table_name AS table_reference
FROM information_schema.table_constraints AS tc
JOIN information_schema.key_column_usage AS kcu ON tc.constraint_name = kcu.constraint_name
JOIN information_schema.constraint_column_usage AS ccu ON ccu.constraint_name = tc.constraint_name
WHERE tc.constraint_type = 'FOREIGN KEY';"
 table_name | column_name | table_reference
-----
progressions | utilisateur_id | utilisateurs
progressions | formation_id | formations
resultats_examens | utilisateur_id | utilisateurs
resultats_examens | formation_id | formations
(4 rows)
```

A2 – Inventaire des fichiers

Fichier	Type	Taille	Critique ?
excel_module1.pdf	PDF	32 B	Oui
cyber_intro.pdf	PDF	37 B	Oui
gestion_methodes.pdf	PDF	37 B	Oui
linux_video1.mp4	Vidéo	33 B	Oui
logo.png	Image	20 B	Non

```
amine@OrdiDeAmine:~/td_migration$ ls -lh data_montpellier/supports/
find data_montpellier/supports/ -type f | wc -l
du -sh data_montpellier/supports/
total 20K
-rw-r--r-- 1 amine amine 37 May 19 11:19 cyber_intro.pdf
-rw-r--r-- 1 amine amine 32 May 19 11:19 excel_module1.pdf
-rw-r--r-- 1 amine amine 37 May 19 11:19 gestion_methodes.pdf
-rw-r--r-- 1 amine amine 33 May 19 11:19 linux_video1.mp4
-rw-r--r-- 1 amine amine 20 May 19 11:19 Logo.png
5
24K data_montpellier/supports/
```

A3 – Identification des contraintes

Bande passante : Illimitée (réseau Docker bridge local).

Durée maximale de coupure : 2 heures selon l'énoncé.

Tables à migrer dans un ordre contraint : progressions et resultats_examens ne peuvent pas être migrées avant utilisateurs et formations, car elles référencent ces tables via des clés étrangères.

Si on migre progressions avant utilisateurs : PostgreSQL lève une erreur de violation de contrainte FK — l'INSERT est rejeté car les utilisateur_id référencés n'existent pas encore sur la cible.

A4 – Ordre de migration des tables

Ordre	Table à migrer	Raison
1	utilisateurs	Aucune dépendance — table racine
2	formations	Aucune dépendance — table racine
3	progressions	Dépend de utilisateurs (FK) et formations (FK)
4	resultats_examens	Dépend de utilisateurs (FK) et formations (FK)

3. Niveau 1 – Migration Big Bang

Principe : Arrêt total du service source → export complet → transfert → restauration complète.
Méthode la plus simple mais implique une interruption totale pendant toute la durée du transfert.

Scripts réalisés

01_arret_source.sh

```
#!/bin/bash

# niveau1_bigbang/01_arret_source.sh

echo '=== [NIVEAU 1] Arrêt du service source ==='

echo "Heure de debut de coupure : $(date)" | tee logs/coupure_debut.log

# Passer la base en lecture seule

docker exec pg_montpellier psql -U admin -d techcorp_db -c \

    "ALTER DATABASE techcorp_db SET default_transaction_read_only = on;"

echo 'OK : Base source passee en lecture seule'

# Verification qu'aucune transaction n'est en cours

docker exec pg_montpellier psql -U admin -d techcorp_db -c \

    "SELECT pid, state, query FROM pg_stat_activity WHERE state = 'active';"
```

```
amine@OrdiDeAmine:~/td_migration$ nano ~/td_migration/niveau1_bigbang/01_arret_source.sh
amine@OrdiDeAmine:~/td_migration$ chmod +x ~/td_migration/niveau1_bigbang/01_arret_source.sh
cd ~/td_migration
bash niveau1_bigbang/01_arret_source.sh
=== [NIVEAU 1] Arrêt du service source ===
Heure de debut de coupure : Tue May 19 11:22:46 CEST 2026
ALTER DATABASE
OK : Base source passee en lecture seule
pid | state | query
-----+-----+-----
151 | active | SELECT pid, state, query FROM pg_stat_activity WHERE state = 'active';
(1 row)
```

02_export_source.sh

```
#!/bin/bash

# niveau1_bigbang/02_export_source.sh

echo '=== [NIVEAU 1] Export de la base source ==='

BACKUP_DIR='./backups'

BACKUP_FILE="$BACKUP_DIR/techcorp_bigbang_$(date +%Y%m%d_%H%M%S).dump"

mkdir -p $BACKUP_DIR
```

```
# Export complet en format custom

docker exec pg_montpellier pg_dump \

-U admin -d techcorp_db \

-F c -v \

-f /tmp/techcorp_backup.dump

# Copier le fichier hors du container

docker cp pg_montpellier:/tmp/techcorp_backup.dump $BACKUP_FILE

echo "OK : Sauvegarde creee : $BACKUP_FILE"

ls -lh $BACKUP_FILE

# Verification que le fichier n'est pas vide

[ -s $BACKUP_FILE ] && echo 'OK : fichier non vide' || echo 'ERREUR : fichier vide'
```

```
11:02
amine@Ordinaire:~/td_migration$ nano ~/td_migration/niveaul_bigbang/02_export_source.sh
amine@Ordinaire:~/td_migration$ bash niveaul_bigbang/02_export_source.sh
=== [NIVEAU 1] Export de la base source ===
pg_dump: last built-in OID is 16383
pg_dump: reading extensions
pg_dump: reading extension members
pg_dump: reading schemas
pg_dump: reading user-defined tables
pg_dump: reading user-defined functions
pg_dump: reading user-defined types
pg_dump: reading procedural languages
pg_dump: reading user-defined aggregate functions
pg_dump: reading user-defined operators
pg_dump: reading user-defined access methods
pg_dump: reading user-defined operator classes
pg_dump: reading user-defined operator families
pg_dump: reading user-defined text search parsers
pg_dump: reading user-defined text search templates
pg_dump: reading user-defined text search dictionaries
pg_dump: reading user-defined text search configurations
pg_dump: reading user-defined foreign-data wrappers
pg_dump: reading user-defined foreign servers
pg_dump: reading default privileges
pg_dump: reading user-defined collations
pg_dump: reading user-defined conversions
pg_dump: reading type casts
pg_dump: reading transforms
pg_dump: reading table inheritance information
pg_dump: reading event triggers
pg_dump: finding extension tables
pg_dump: finding inheritance relationships
pg_dump: reading column info for interesting tables
pg_dump: finding table default expressions
pg_dump: flagging inherited columns in subtables
pg_dump: reading partitioning data
pg_dump: reading indexes
pg_dump: flagging indexes in partitioned tables
pg_dump: reading extended statistics
pg_dump: reading constraints
pg_dump: reading triggers
pg_dump: reading rewrite rules
pg_dump: reading policies
pg_dump: reading row-level security policies
pg_dump: reading publications
pg_dump: reading publication membership of tables
pg_dump: reading publication membership of schemas
pg_dump: reading subscriptions
pg_dump: reading large objects
pg_dump: reading dependency data
pg_dump: saving encoding = utf8
pg_dump: saving standard_conforming_strings = on
pg_dump: saving search_path =
pg_dump: saving database definition
pg_dump: dumping contents of table "public.formations"
pg_dump: dumping contents of table "public.progressions"
pg_dump: dumping contents of table "public.resultats_examens"
pg_dump: dumping contents of table "public.utilisateurs"
Successfully copied 13.8kB to /home/amine/td_migration/backups/techcorp_bigbang_20260519_112346.dump
OK : Sauvegarde creee : /backups/techcorp_bigbang_20260519_112346.dump
-rw-r--r-- 1 amine amine 12K May 19 11:23 /backups/techcorp_bigbang_20260519_112346.dump
on : fichier non vide
```

03_transfert_fichiers.sh

```
#!/bin/bash

# niveaul_bigbang/03_transfert_fichiers.sh

echo '=== [NIVEAU 1] Transfert des fichiers ==='

SOURCE='./data_montpellier/supports/'

CIBLE='./data_toulouse/supports/'

rsync -avh --progress --checksum $SOURCE $CIBLE

echo ''
```

```

echo '=== Verification du transfert ==='

NB_SOURCE=$(find $SOURCE -type f | wc -l)
NB_CIBLE=$(find $CIBLE -type f | wc -l)

echo "Fichiers source : $NB_SOURCE"

echo "Fichiers cible : $NB_CIBLE"

if [ $NB_SOURCE -eq $NB_CIBLE ]; then

    echo 'OK : meme nombre de fichiers'

else

    echo 'ERREUR : nombre de fichiers different !'

    exit 1

fi

# Verification par checksums MD5

md5sum data_montpellier/supports/* > /tmp/checksums_source.txt

md5sum data_toulouse/supports/* > /tmp/checksums_cible.txt

# Comparer uniquement les hashes (pas les chemins)

awk '{print $1}' /tmp/checksums_source.txt | sort > /tmp/hash_source.txt

awk '{print $1}' /tmp/checksums_cible.txt | sort > /tmp/hash_cible.txt

diff /tmp/hash_source.txt /tmp/hash_cible.txt && echo 'OK : checksums identiques' || echo 'ERREUR : checksums
different'

```

```

amine@OrdiDeAmine:~/td_migration$ nano ~/td_migration/niveau1_bigbang/03_transfert_fichiers.sh
amine@OrdiDeAmine:~/td_migration$ bash niveau1_bigbang/03_transfert_fichiers.sh
=== [NIVEAU 1] Transfert des fichiers ===
sending incremental file list
./
cyber_intro.pdf      37 100%   0.00kB/s   0:00:00 (xfr#1, to-chk=4/6)
excel_module1.pdf    32 100%  31.25kB/s   0:00:00 (xfr#2, to-chk=3/6)
gestion_methodes.pdf 37 100%  36.13kB/s   0:00:00 (xfr#3, to-chk=2/6)
linux_video1.mp4     33 100%  32.23kB/s   0:00:00 (xfr#4, to-chk=1/6)
logo.png             20 100%  19.53kB/s   0:00:00 (xfr#5, to-chk=0/6)

sent 651 bytes  received 114 bytes  1.53K bytes/sec
total size is 159  speedup is 0.21

=== Verification du transfert ===
Fichiers source : 5
Fichiers cible : 5
OK : meme nombre de fichiers
OK : checksums identiques

```

04_restauracion_cible.sh

```

#!/bin/bash

# niveau1_bigbang/04_restauracion_cible.sh

echo '=== [NIVEAU 1] Restauration sur la cible (Toulouse) ==='

BACKUP_FILE=$(ls -t ./backups/techcorp_bigbang_*.dump | head -1)

if [ -z "$BACKUP_FILE" ]; then

```

```

    echo 'ERREUR : aucun fichier de sauvegarde trouve !'

    exit 1

fi

echo "Fichier utilise : $BACKUP_FILE"

# Copier le dump dans le container cible

docker cp $BACKUP_FILE pg_toulouse:/tmp/techcorp_backup.dump

# Restaurer la base sur la cible

docker exec pg_toulouse pg_restore \

    -U admin -d techcorp_db \

    -v --clean --if-exists \

    /tmp/techcorp_backup.dump

echo 'OK : Restauration terminee'

# Verification du nombre de tables restaurees

docker exec pg_toulouse psql -U admin -d techcorp_db -c '\dt'

```

```

OK : checksums identiques
amine@ordi05Amine:~/td_migration$ nano ~/td_migration/niveau_bigbang/04_restoration_cible.sh
amine@ordi05Amine:~/td_migration$ bash niveau_bigbang/04_restoration_cible.sh
=== [NIVEAU 1] Restauration sur la cible (toulouse) ===
Fichier utilise : ./backups/techcorp_bigbang_20200519_112346.dump
Successfully copied 13.8kB to pg_toulouse:/tmp/techcorp_backup.dump
pg_restore: connecting to database for restore
pg_restore: dropping FK CONSTRAINT resultats_examens resultats_examens_utilisateur_id_fkey
pg_restore: dropping FK CONSTRAINT resultats_examens resultats_examens_formation_id_fkey
pg_restore: dropping FK CONSTRAINT progressions progressions_utilisateur_id_fkey
pg_restore: dropping FK CONSTRAINT progressions progressions_formation_id_fkey
pg_restore: dropping CONSTRAINT utilisateurs utilisateurs_pkey
pg_restore: dropping CONSTRAINT utilisateurs utilisateurs_email_key
pg_restore: dropping CONSTRAINT resultats_examens resultats_examens_pkey
pg_restore: dropping CONSTRAINT progressions progressions_pkey
pg_restore: dropping CONSTRAINT formations formations_pkey
pg_restore: dropping DEFAULT utilisateurs id
pg_restore: dropping DEFAULT resultats_examens id
pg_restore: dropping DEFAULT progressions id
pg_restore: dropping DEFAULT formations id
pg_restore: dropping SEQUENCE utilisateurs_id_seq
pg_restore: dropping TABLE utilisateurs
pg_restore: dropping SEQUENCE resultats_examens_id_seq
pg_restore: dropping TABLE resultats_examens
pg_restore: dropping SEQUENCE progressions_id_seq
pg_restore: dropping TABLE progressions
pg_restore: dropping SEQUENCE formations_id_seq
pg_restore: dropping TABLE formations
pg_restore: creating TABLE "public.formations"
pg_restore: creating SEQUENCE "public.formations_id_seq"
pg_restore: creating SEQUENCE OWNED BY "public.formations_id_seq"
pg_restore: creating TABLE "public.progressions"
pg_restore: creating SEQUENCE "public.progressions_id_seq"
pg_restore: creating SEQUENCE OWNED BY "public.progressions_id_seq"
pg_restore: creating TABLE "public.resultats_examens"
pg_restore: creating SEQUENCE "public.resultats_examens_id_seq"
pg_restore: creating SEQUENCE OWNED BY "public.resultats_examens_id_seq"
pg_restore: creating TABLE "public.utilisateurs"
pg_restore: creating SEQUENCE "public.utilisateurs_id_seq"
pg_restore: creating SEQUENCE OWNED BY "public.utilisateurs_id_seq"
pg_restore: creating DEFAULT "public.formations id"
pg_restore: creating DEFAULT "public.progressions id"
pg_restore: creating DEFAULT "public.resultats_examens id"
pg_restore: creating DEFAULT "public.utilisateurs id"
pg_restore: processing data for table "public.formations"
pg_restore: processing data for table "public.progressions"
pg_restore: processing data for table "public.resultats_examens"
pg_restore: processing data for table "public.utilisateurs"
pg_restore: executing SEQUENCE SET formations_id_seq
pg_restore: executing SEQUENCE SET progressions_id_seq
pg_restore: executing SEQUENCE SET resultats_examens_id_seq
pg_restore: executing SEQUENCE SET utilisateurs_id_seq
pg_restore: creating CONSTRAINT "public.formations_formations_pkey"
pg_restore: creating CONSTRAINT "public.progressions progressions_pkey"
pg_restore: creating CONSTRAINT "public.resultats_examens resultats_examens_pkey"
pg_restore: creating CONSTRAINT "public.utilisateurs utilisateurs_email_key"
pg_restore: creating CONSTRAINT "public.utilisateurs utilisateurs_pkey"
pg_restore: creating FK CONSTRAINT "public.progressions progressions_formation_id_fkey"
pg_restore: creating FK CONSTRAINT "public.progressions progressions_utilisateur_id_fkey"
pg_restore: creating FK CONSTRAINT "public.resultats_examens resultats_examens_formation_id_fkey"
pg_restore: creating FK CONSTRAINT "public.resultats_examens resultats_examens_utilisateur_id_fkey"
OK : Restauration terminee
List of relations

```

Schema	Name	Type	Owner
public	formations	table	admin
public	progressions	table	admin
public	resultats_examens	table	admin
public	utilisateurs	table	admin

```

(4 rows)

```


05_verification.sh

```
#!/bin/bash

# niveau1_bigbang/05_verification.sh

echo '=== [NIVEAU 1] Verification de l integrite ==='

echo '--- Comptage SOURCE (Montpellier) ---'

docker exec pg_montpellier psql -U admin -d techcorp_db -t -c \

    "SELECT 'utilisateurs', COUNT(*) FROM utilisateurs

    UNION ALL SELECT 'formations', COUNT(*) FROM formations

    UNION ALL SELECT 'progressions', COUNT(*) FROM progressions

    UNION ALL SELECT 'resultats_examens', COUNT(*) FROM resultats_examens;"

echo '--- Comptage CIBLE (Toulouse) ---'

docker exec pg_toulouse psql -U admin -d techcorp_db -t -c \

    "SELECT 'utilisateurs', COUNT(*) FROM utilisateurs

    UNION ALL SELECT 'formations', COUNT(*) FROM formations

    UNION ALL SELECT 'progressions', COUNT(*) FROM progressions

    UNION ALL SELECT 'resultats_examens', COUNT(*) FROM resultats_examens;"

echo "Heure de fin : $(date)" | tee -a logs/coupure_debut.log

# Duree totale de coupure

DEBUT=$(grep "debut" logs/coupure_debut.log | head -1 | awk -F': ' '{print $2}')

echo "Debut coupure : $DEBUT"

echo "Fin coupure : $(date)"

# Verification FK sur la cible

echo '--- Verification des cles etrangeres sur la cible ---'

docker exec pg_toulouse psql -U admin -d techcorp_db -c \

    "SELECT tc.table_name, kcu.column_name, ccu.table_name AS table_referencee

    FROM information_schema.table_constraints AS tc

    JOIN information_schema.key_column_usage AS kcu ON tc.constraint_name = kcu.constraint_name

    JOIN information_schema.constraint_column_usage AS ccu ON ccu.constraint_name = tc.constraint_name

    WHERE tc.constraint_type = 'FOREIGN KEY';"
```

```

amine@OrdiDeAmine:~/td_migration$ nano ~/td_migration/niveau1_bigbang/05_verification.sh
amine@OrdiDeAmine:~/td_migration$ bash niveau1_bigbang/05_verification.sh
=== [NIVEAU 1] Verification de l integrite ===
--- Comptage SOURCE (Montpellier) ---
utilisateurs | 5
formations   | 4
progressions | 6
resultats_examens | 3

--- Comptage CIBLE (Toulouse) ---
utilisateurs | 5
formations   | 4
progressions | 6
resultats_examens | 3

Heure de fin : Tue May 19 11:26:51 CEST 2026
Debut coupure : Tue May 19 11:22:46 CEST 2026
Fin coupure : Tue May 19 11:26:51 CEST 2026
--- Verification des cles etrangeres sur la cible ---
table_name | column_name | table.reference
-----
progressions | formation_id | formations
progressions | utilisateur_id | utilisateurs
resultats_examens | formation_id | formations
resultats_examens | utilisateur_id | utilisateurs
(4 rows)

```

06_rollback.sh

```

#!/bin/bash

# niveau1_bigbang/06_rollback.sh

echo '=== [NIVEAU 1] ROLLBACK - Retour a Montpellier ==='

echo 'ATTENTION : Cette operation annule la migration !'

read -p 'Confirmer le rollback ? (oui/non) : ' CONFIRM

if [ "$CONFIRM" != 'oui' ]; then

    echo 'Rollback annule.'

    exit 0

fi

# Remettre la source en lecture/ecriture

docker exec pg_montpellier psql -U admin -d techcorp_db -c \

    "ALTER DATABASE techcorp_db SET default_transaction_read_only = off;"

echo 'OK : Base source remise en lecture/ecriture'

# Vider la base cible pour eviter toute confusion

docker exec pg_toulouse psql -U admin -d techcorp_db -c \

    "DROP TABLE IF EXISTS progressions, resultats_examens, utilisateurs, formations CASCADE;"

echo 'OK : Base cible videe'

# Logger la raison du rollback

echo "$(date) - Rollback N1 effectue" >> logs/rollback.log

echo 'OK : Rollback termine, service reprend sur Montpellier'

```

```

amine@OrdiDeAmine:~/td_migration$ nano ~/td_migration/niveau1_bigbang/06_rollback.sh
amine@OrdiDeAmine:~/td_migration$ bash niveau1_bigbang/06_rollback.sh
=== [NIVEAU 1] ROLLBACK - Retour a Montpellier ===
ATTENTION : Cette operation annule la migration !
Confirmer le rollback ? (oui/non) : o
Rollback annule.
amine@OrdiDeAmine:~/td_migration$ docker exec pg_montpellier psql -U admin -d techcorp_db -c \
    "ALTER DATABASE techcorp_db SET default_transaction_read_only = off;"
ERROR: cannot execute ALTER DATABASE in a read-only transaction
amine@OrdiDeAmine:~/td_migration$ docker exec pg_montpellier psql -U admin -d postgres -c \
    "ALTER DATABASE techcorp_db SET default_transaction_read_only = off;"
ALTER DATABASE
amine@OrdiDeAmine:~/td_migration$ docker exec pg_montpellier psql -U admin -d techcorp_db -c \
    "INSERT INTO utilisateurs (nom, email) VALUES ('Test RW', 'testrw@techcorp.fr');"
INSERT 0 1

```

Résultat final — Vérification d'intégrité

Table	Lignes SOURCE	Lignes CIBLE	Identiques ?
utilisateurs	5	5	Oui
formations	4	4	Oui
progressions	6	6	Oui
resultats_examens	3	3	Oui

Durée de coupure mesurée : 11:22:46 → 11:26:51 = ~4 minutes

Questions d'analyse – Niveau 1

Q1 : Combien de temps a duré la coupure ? Était-ce dans la fenêtre des 2 heures ?

La coupure a duré environ 4 minutes (11:22:46 -> 11:26:51). C'est largement dans la fenêtre autorisée de 2 heures. En production réelle avec une base volumineuse (dizaines de Go), la durée serait bien plus longue — potentiellement plusieurs heures.

Q2 : Quels sont les risques d'une migration Big Bang sur une base de production réelle ?

- Interruption totale du service pendant toute la durée du transfert — aucun utilisateur ne peut travailler.
- Aucune possibilité de retour en arrière rapide si la restauration échoue sans rollback préparé.
- Fenêtre de risque maximale : plus la base est grosse, plus la coupure est longue.
- Pression temporelle : si la restauration dépasse la fenêtre, le service reste indisponible.

Q3 : Que se passe-t-il si la restauration échoue à mi-chemin ?

La base cible se retrouve dans un état incohérent (tables partiellement créées, données incomplètes). Il faut relancer la restauration depuis zéro ou déclencher le rollback pour remettre la source en production. C'est pour cela que le script 06_rollback.sh est indispensable.

Q4 : Pourquoi l'ordre de restauration des tables est-il important ?

À cause des contraintes de clés étrangères. Si on insère des lignes dans 'progressions' avant que 'utilisateurs' et 'formations' existent sur la cible, PostgreSQL rejette les insertions avec une erreur FK. L'outil pg_restore gère cet ordre automatiquement grâce au fichier .dump.

Q5 : Dans quel cas réel pourrait-on accepter une migration Big Bang ?

- Base de données de taille modeste pouvant être transférée en quelques minutes.
- Service disposant d'une fenêtre de maintenance nocturne longue.
- Projet en cours de développement, pas encore en production.
- Quand la simplicité est prioritaire sur la disponibilité.

4. Niveau 2 – Migration Progressive

Principe : Synchronisation préalable hors coupure (la majorité des données), puis courte coupure finale pour transférer uniquement le delta. Réduit drastiquement la durée d'interruption.

Scripts réalisés

01_sync_initiale.sh

```
#!/bin/bash

# niveau2_progressive/01_sync_initiale.sh

echo '=== [NIVEAU 2] Synchronisation initiale (hors coupure) ==='

echo "Debut : $(date)"

BACKUP_DIR='./backups'

mkdir -p $BACKUP_DIR

# Export de la base source en format SQL plain
docker exec pg_montpellier pg_dump \

    -U admin -d techcorp_db \

    --format=plain --no-owner --no-acl \

    -f /tmp/sync_initiale.sql

docker cp pg_montpellier:/tmp/sync_initiale.sql $BACKUP_DIR/sync_initiale.sql

# Restaurer sur la cible (vider d'abord)
docker exec pg_toulouse psql -U admin -d techcorp_db -c \

    "DROP TABLE IF EXISTS progressions, resultats_examens, utilisateurs, formations CASCADE;"

docker cp $BACKUP_DIR/sync_initiale.sql pg_toulouse:/tmp/sync_initiale.sql

docker exec pg_toulouse psql -U admin -d techcorp_db -f /tmp/sync_initiale.sql

# Synchronisation des fichiers
rsync -avh --progress \

    ./data_montpellier/supports/ \

    ./data_toulouse/supports/
```

```
echo "OK : Synchronisation initiale terminee : $(date)"

# Checksum de l'etat de la base

docker exec pg_montpellier psql -U admin -d techcorp_db -t -c \

    "SELECT COUNT(*) FROM utilisateurs;" > ./backups/checksum_sync_initiale.txt

echo "Checksum enregistre : $(cat ./backups/checksum_sync_initiale.txt)"
```

[illegible]

02_simulation_activite.sh

```
#!/bin/bash

# niveau2_progressive/02_simulation_activite.sh

echo '=== [NIVEAU 2] Simulation d activite utilisateur ==='


# Ajouter de nouveaux utilisateurs

docker exec pg_montpellier psql -U admin -d techcorp_db -c \

"INSERT INTO utilisateurs (nom, email) VALUES

('Francois Petit', 'francois@techcorp.fr'),

('Gaelle Simon', 'gaelle@techcorp.fr');"


# Mettre a jour une progression

docker exec pg_montpellier psql -U admin -d techcorp_db -c \

"UPDATE progressions SET pourcentage = 100, derniere_activite = NOW()"
```

```

WHERE utilisateur_id = 1 AND formation_id = 2;"

# Ajouter un resultat d'examen

docker exec pg_montpellier psql -U admin -d techcorp_db -c \

    "INSERT INTO resultats_examens (utilisateur_id, formation_id, note)

    VALUES (3, 1, 16.5);"

# Ajouter un nouveau fichier support

echo 'Nouveau support Python debutants' > data_montpellier/supports/python_intro.pdf

echo 'OK : Nouvelles donnees ajoutees sur la source'

echo '  2 nouveaux utilisateurs, 1 progression mise a jour'

echo '  1 nouveau resultat examen, 1 nouveau fichier support'

# Verification

docker exec pg_montpellier psql -U admin -d techcorp_db -c \

    "SELECT COUNT(*) AS nb_utilisateurs FROM utilisateurs;"

```

```

amine@OrdiDeAmine:~/td_migration$ nano ~/td_migration/niveau2_progressive/02_simulation_activite.sh
amine@OrdiDeAmine:~/td_migration$ bash niveau2_progressive/02_simulation_activite.sh
=== [NIVEAU 2] Simulation d activite utilisateur ===
INSERT 0 2
UPDATE 1
INSERT 0 1
OK : Nouvelles donnees ajoutees sur la source
  2 nouveaux utilisateurs, 1 progression mise a jour
  1 nouveau resultat examen, 1 nouveau fichier support
nb_utilisateurs
-----
      8
(1 row)

```

03_sync_différentielle.sh

```

#!/bin/bash

# niveau2_progressive/03_sync_différentielle.sh

echo '=== [NIVEAU 2] Synchronisation différentielle ==='

# Fichiers : rsync ne retransfere que les nouveaux/modifies

rsync -avh --progress --checksum \

    ./data_montpellier/supports/ \

    ./data_toulouse/supports/

# Base : export des donnees recentes

docker exec pg_montpellier pg_dump \

    -U admin -d techcorp_db \

    --format=plain --data-only \

```

```

-f /tmp/sync_diff.sql

docker cp pg_montpellier:/tmp/sync_diff.sql ./backups/sync_diff.sql

echo 'OK : Synchronisation differentielle terminee'

# Comparer le nombre de lignes entre source et cible

echo '--- Comparaison source vs cible ---'

echo -n 'Source utilisateurs : '

docker exec pg_montpellier psql -U admin -d techcorp_db -t -c 'SELECT COUNT(*) FROM utilisateurs;'

echo -n 'Cible utilisateurs : '

docker exec pg_toulouse psql -U admin -d techcorp_db -t -c 'SELECT COUNT(*) FROM utilisateurs;'

```

```

amine@OrdiDeAmine:~/td_migration$ nano ~/td_migration/niveau2_progressive/03_sync_differentielle.sh
amine@OrdiDeAmine:~/td_migration$ bash niveau2_progressive/03_sync_differentielle.sh
=== [NIVEAU 2] Synchronisation differentielle ===
sending incremental file list
./
python_intro.pdf
 33 100%  0.00kB/s  0:00:00 (xfr#1, to-chk=0/7)

sent 406 bytes  received 38 bytes  888.00 bytes/sec
total size is 192  speedup is 0.43
Successfully copied 4.61kB to /home/amine/td_migration/backups/sync_diff.sql
OK : Synchronisation differentielle terminee
--- Comparaison source vs cible ---
Source utilisateurs :      8
Cible utilisateurs :      6

```

04_coupure_finale.sh

```

#!/bin/bash

# niveau2_progressive/04_coupure_finale.sh

echo '=== [NIVEAU 2] COUPURE - Synchronisation finale ==='

echo "Debut de la fenetre de coupure : $(date)" | tee logs/coupure_n2.log

# 1. Passer la source en lecture seule

docker exec pg_montpellier psql -U admin -d postgres -c \

    "ALTER DATABASE techcorp_db SET default_transaction_read_only = on;"

echo 'OK : Source en lecture seule'

# 2. Derniere synchronisation des fichiers

rsync -avh --checksum --delete \

    ./data_montpellier/supports/ \

    ./data_toulouse/supports/

echo 'OK : Fichiers synchronises'

# 3. Dernier export de la base

docker exec pg_montpellier pg_dump \

    -U admin -d techcorp_db \

```

```
--format=custom -f /tmp/sync_finale.dump

docker cp pg_montpellier:/tmp/sync_finale.dump ./backups/sync_finale.dump

echo 'OK : Export final cree'

# 4. Restauration finale sur la cible

docker exec pg_toulouse psql -U admin -d techcorp_db -c \

"DROP TABLE IF EXISTS progressions, resultats_examens, utilisateurs, formations CASCADE;"

docker cp ./backups/sync_finale.dump pg_toulouse:/tmp/sync_finale.dump

docker exec pg_toulouse pg_restore \

-U admin -d techcorp_db \

--clean --if-exists -v \

/tmp/sync_finale.dump

echo 'OK : Restauration finale terminee'

echo "Fin de la synchronisation finale : $(date)" | tee -a logs/coupure_n2.log

# Duree de coupure
DEBUT=$(grep "Debut" logs/coupure_n2.log | awk -F': ' '{print $2}')

echo "Debut : $DEBUT"

echo "Fin : $(date)"
```

```
aine@0rdiDeAine:~/td_migration$ nano ~/td_migration/niveau2_progressive/04_coupure_finale.sh
aine@0rdiDeAine:~/td_migration$ bash niveau2_progressive/04_coupure_finale.sh
== [NIVEAU 2] COUPURE - Synchronisation finale ==
Debut de la fenetre de coupure : Tue May 19 11:35:36 CEST 2026
ALTER DATABASE
ON : Source en lecture seule
sending incremental file list

sent 323 bytes received 12 bytes 670.00 bytes/sec
total size is 192 speedup is 0.57
OK : Fichiers synchronises
Successfully copied 13.8k to /home/aine/td_migration/backups/sync_finale.dump
OK : Export final cree
DROP TABLE
Successfully copied 13.8k to pg_toulouse:/tmp/sync_finale.dump
pg_restore: connecting to database for restore
pg_restore: dropping FK CONSTRAINT resultats_examens_resultats_examens_utilisateur_id_fkey
pg_restore: dropping FK CONSTRAINT resultats_examens_resultats_examens_formation_id_fkey
pg_restore: dropping FK CONSTRAINT progressions_progressions_utilisateur_id_fkey
pg_restore: dropping FK CONSTRAINT progressions_progressions_formation_id_fkey
pg_restore: dropping CONSTRAINT utilisateurs_utilisateurs_pkey
pg_restore: dropping CONSTRAINT utilisateurs_utilisateurs_email_key
pg_restore: dropping CONSTRAINT resultats_examens_resultats_examens_pkey
pg_restore: dropping CONSTRAINT progressions_progressions_pkey
pg_restore: dropping CONSTRAINT formations_formations_pkey
pg_restore: dropping DEFAULT utilisateurs_id
pg_restore: dropping DEFAULT resultats_examens_id
pg_restore: dropping DEFAULT progressions_id
pg_restore: dropping DEFAULT formations_id
pg_restore: dropping SEQUENCE utilisateurs_id_seq
pg_restore: dropping TABLE utilisateurs
pg_restore: dropping SEQUENCE resultats_examens_id_seq
pg_restore: dropping TABLE resultats_examens
pg_restore: dropping SEQUENCE progressions_id_seq
pg_restore: dropping TABLE progressions
pg_restore: dropping SEQUENCE formations_id_seq
pg_restore: dropping TABLE formations
pg_restore: creating TABLE "public.formations"
pg_restore: creating SEQUENCE "public.formations_id_seq"
pg_restore: creating SEQUENCE OWNED BY "public.formations_id_seq"
pg_restore: creating TABLE "public.progressions"
pg_restore: creating SEQUENCE "public.progressions_id_seq"
pg_restore: creating SEQUENCE OWNED BY "public.progressions_id_seq"
pg_restore: creating TABLE "public.resultats_examens"
pg_restore: creating SEQUENCE "public.resultats_examens_id_seq"
pg_restore: creating SEQUENCE OWNED BY "public.resultats_examens_id_seq"
pg_restore: creating TABLE "public.utilisateurs"
pg_restore: creating SEQUENCE "public.utilisateurs_id_seq"
pg_restore: creating SEQUENCE OWNED BY "public.utilisateurs_id_seq"
pg_restore: creating DEFAULT "public.formations_id"
pg_restore: creating DEFAULT "public.progressions_id"
pg_restore: creating DEFAULT "public.resultats_examens_id"
pg_restore: processing data for table "public.formations"
pg_restore: processing data for table "public.progressions"
pg_restore: processing data for table "public.resultats_examens"
pg_restore: processing data for table "public.utilisateurs"
pg_restore: executing SEQUENCE SET formations_id_seq
pg_restore: executing SEQUENCE SET progressions_id_seq
pg_restore: executing SEQUENCE SET resultats_examens_id_seq
pg_restore: creating CONSTRAINT "public.formations_formations_pkey"
pg_restore: creating CONSTRAINT "public.progressions_progressions_pkey"
pg_restore: creating CONSTRAINT "public.resultats_examens_resultats_examens_pkey"
pg_restore: creating CONSTRAINT "public.utilisateurs_utilisateurs_email_key"
pg_restore: creating CONSTRAINT "public.utilisateurs_utilisateurs_pkey"
pg_restore: creating FK CONSTRAINT "public.progressions_progressions_formation_id_fkey"
pg_restore: creating FK CONSTRAINT "public.progressions_progressions_utilisateur_id_fkey"
pg_restore: creating FK CONSTRAINT "public.resultats_examens_resultats_examens_formation_id_fkey"
pg_restore: creating FK CONSTRAINT "public.resultats_examens_resultats_examens_utilisateur_id_fkey"
OK : Restauration finale terminee
Fin de la synchronisation finale : Tue May 19 11:35:38 CEST 2026
Debut : Tue May 19 11:35:36 CEST 2026
Fin : Tue May 19 11:35:38 CEST 2026
```


05_basculer_validation.sh

```
#!/bin/bash

# niveau2_progressive/05_basculer_validation.sh

echo '=== [NIVEAU 2] Basculer et validation ==='

SOURCE_COUNT=$(docker exec pg_montpellier psql -U admin -d techcorp_db -t -c \
    'SELECT COUNT(*) FROM utilisateurs;' | tr -d ' ')

CIBLE_COUNT=$(docker exec pg_toulouse psql -U admin -d techcorp_db -t -c \
    'SELECT COUNT(*) FROM utilisateurs;' | tr -d ' ')

echo "Utilisateurs source : $SOURCE_COUNT"
echo "Utilisateurs cible : $CIBLE_COUNT"

if [ "$SOURCE_COUNT" != "$CIBLE_COUNT" ]; then
    echo 'ERREUR : les comptages ne correspondent pas !'
    echo 'Rollback recommande'
    exit 1
fi

echo 'OK : Comptages identiques - basculer autorisee'

# Activer la cible en lecture/ecriture
docker exec pg_toulouse psql -U admin -d techcorp_db -c \
    "ALTER DATABASE techcorp_db SET default_transaction_read_only = off;"
echo 'OK : Cible activee en lecture/ecriture'

# Test insertion sur la cible
docker exec pg_toulouse psql -U admin -d techcorp_db -c \
    "INSERT INTO utilisateurs (nom, email) VALUES ('Test Basculer N2', 'testN2@techcorp.fr');"
echo 'OK : Insertion test sur la cible reussie'

echo 'OK : Migration progressive terminee avec succes'

echo "Heure de fin : $(date)" | tee -a logs/coupure_n2.log
```

```
Fin : Tue May 19 11:35:38 CEST 2026
amine@OrdideAmine:~/td_migration$ nano ~/td_migration/niveau2_progressive/05_basculer_validation.sh
amine@OrdideAmine:~/td_migration$ bash niveau2_progressive/05_basculer_validation.sh
=== [NIVEAU 2] Basculer et validation ===
Utilisateurs source : 8
Utilisateurs cible : 8
OK : Comptages identiques - basculer autorisee
ALTER DATABASE
OK : Cible activee en lecture/ecriture
INSERT 0 1
OK : Insertion test sur la cible reussie
OK : Migration progressive terminee avec succes
Heure de fin : Tue May 19 11:36:26 CEST 2026
amine@OrdideAmine:~/td_migration$ docker exec pg_montpellier psql -U admin -d postgres -c \
    "ALTER DATABASE techcorp_db SET default_transaction_read_only = off;"
ALTER DATABASE
```

Résultat final – Durée de coupure N2

Début coupure : 11:35:36 | Fin coupure : 11:35:38 | Durée : ~2 secondes

Utilisateurs source = cible = 8 lignes — Migration progressive réussie.

Questions d'analyse – Niveau 2

Q1 : Comparez la durée de coupure entre N1 et N2.

N1 Big Bang : ~4 minutes de coupure totale — toute la base est transférée pendant l'interruption.
N2 Progressive : ~2 secondes de coupure effective — la quasi-totalité des données avait déjà été synchronisée en avance. Sur une vraie base de 100 Go, la différence serait encore plus spectaculaire.

Q2 : Pourquoi l'option --delete de rsync est-elle importante lors de la synchronisation finale ?

L'option --delete supprime sur la cible les fichiers qui ont été effacés sur la source entre les deux synchronisations. Sans cette option, des fichiers obsolètes resteraient présents sur la cible, créant une divergence. C'est essentiel pour garantir que la cible est une image fidèle de la source.

Q3 : Que se passe-t-il si un utilisateur s'inscrit entre la sync initiale et la coupure ?

Cet utilisateur existe sur la source mais pas encore sur la cible après la sync initiale. Il sera rattrapé lors de la synchronisation finale (pendant la coupure), qui exporte et restaure l'intégralité de la base. Aucune donnée n'est perdue à condition que la coupure finale soit faite proprement.

Q4 : Pourquoi vérifie-t-on les comptages AVANT de basculer ?

Pour s'assurer que la restauration finale s'est déroulée correctement et que source et cible sont cohérentes. Si les comptages diffèrent, cela signale un problème (restauration partielle, erreur FK). Basculer sur une cible incomplète causerait des pertes de données en production.

Q5 : Dans quel cas la migration progressive ne suffit-elle pas ?

- Quand même 2 secondes de coupure sont inacceptables (trading financier, systèmes temps réel).
 - Quand la base est si volumineuse que la synchronisation différentielle prend trop de temps.
 - Quand les données changent en permanence et que le delta reste toujours important.
- Dans ces cas, la réplication logique (Niveau 3) est nécessaire.

5. Niveau 3 – Réplication Logique

Principe : PostgreSQL maintient la cible synchronisée en temps réel via PUBLICATION/SUBSCRIPTION. La cible reçoit chaque modification au fil de l'eau. La bascule se fait à chaud avec une coupure de quelques secondes seulement.

Scripts réalisés

01_verif_config.sh

```
#!/bin/bash

# niveau3_replication/01_verif_config.sh

echo '=== [NIVEAU 3] Verification de la configuration ==='

WAL_LEVEL=$(docker exec pg_montpellier psql -U admin -d postgres -t -c 'SHOW wal_level;' |
tr -d ' \n')

echo "WAL level source : $WAL_LEVEL"

if [ "$WAL_LEVEL" != 'logical' ]; then

    echo 'ERREUR : wal_level doit etre logical pour la replication'

    exit 1

fi

echo 'OK : Configuration WAL correcte'

docker exec pg_montpellier psql -U admin -d postgres -c 'SELECT version();'
```

```
amine@OrdiDeAmine:~/td_migration$ ^C
amine@OrdiDeAmine:~/td_migration$ nano ~/td_migration/niveau3_replication/01_verif_config.sh
amine@OrdiDeAmine:~/td_migration$ bash niveau3_replication/01_verif_config.sh
=== [NIVEAU 3] Verification de la configuration ===
psql: error: connection to server on socket "/var/run/postgresql/.s.PGSQL.5432" failed: FATAL: database "admin" does not exist
WAL level source :
ERREUR : wal_level doit etre logical pour la replication
amine@OrdiDeAmine:~/td_migration$ nano ~/td_migration/niveau3_replication/01_verif_config.sh
amine@OrdiDeAmine:~/td_migration$ bash niveau3_replication/01_verif_config.sh
=== [NIVEAU 3] Verification de la configuration ===
WAL level source : logical
OK : Configuration WAL correcte

version

-----
PostgreSQL 15.18 (Debian 15.18-1.pgdg13+1) on x86_64-pc-linux-gnu, compiled by gcc (Debian 14.2.0-19) 14.2.0, 64-bit
(1 row)
```

02_publication.sh

```
#!/bin/bash

# niveau3_replication/02_publication.sh

echo '=== [NIVEAU 3] Creation de la PUBLICATION sur la source ==='

# Creer une publication pour toutes les tables

docker exec pg_montpellier psql -U admin -d techcorp_db -c \

    "CREATE PUBLICATION pub_techcorp FOR ALL TABLES;"
```

```
# Verifier que la publication est creee

docker exec pg_montpellier psql -U admin -d techcorp_db -c \

    "SELECT pubname, puballtables FROM pg_publication;"

echo 'OK : Publication creee sur la source'

# Afficher les tables publiees

docker exec pg_montpellier psql -U admin -d techcorp_db -c \

    "SELECT * FROM pg_publication_tables;"
```

```
amine@OrdiDeAmine:~/td_migration$ nano ~/td_migration/niveau3_replication/02_publication.sh
amine@OrdiDeAmine:~/td_migration$ bash niveau3_replication/02_publication.sh
=== [NIVEAU 3] Creation de la PUBLICATION sur la source ===
CREATE PUBLICATION
 pubname | puballtables
-----+-----
 pub_techcorp | t
(1 row)

OK : Publication creee sur la source
 pubname | schemaname | tablename | attnames | rowfilter
-----+-----+-----+-----+-----
 pub_techcorp | public | utilisateurs | {id,nom,email,date_inscription} |
 pub_techcorp | public | progressions | {id,utilisateur_id,formation_id,pourcentage,derniere_activite} |
 pub_techcorp | public | formations | {id,titre,categorie,duree_heures} |
 pub_techcorp | public | resultats_examens | {id,utilisateur_id,formation_id,note,date_examen} |
(4 rows)
```

03_structure_cible.sh

```
#!/bin/bash

# niveau3_replication/03_structure_cible.sh

echo '=== [NIVEAU 3] Creation de la structure sur la cible ==='

# Vider la cible d'abord

docker exec pg_toulouse psql -U admin -d techcorp_db -c \

    "DROP TABLE IF EXISTS progressions, resultats_examens, utilisateurs, formations CASCADE;"

# Exporter uniquement le schema

docker exec pg_montpellier pg_dump \

    -U admin -d techcorp_db \

    --schema-only --no-owner \

    -f /tmp/schema_only.sql

docker cp pg_montpellier:/tmp/schema_only.sql ./backups/schema_only.sql

docker cp ./backups/schema_only.sql pg_toulouse:/tmp/schema_only.sql

# Appliquer le schema sur la cible

docker exec pg_toulouse psql -U admin -d techcorp_db -f /tmp/schema_only.sql

echo 'OK : Structure creee sur la cible'
```

```
docker exec pg_toulouse psql -U admin -d techcorp_db -c '\dt'

# Verifier que les tables sont vides

docker exec pg_toulouse psql -U admin -d techcorp_db -c \

'SELECT COUNT(*) AS nb_utilisateurs FROM utilisateurs;'
```

```
amine@OrdideAmine:~/td_migration$ nano ~/td_migration/niveau3_replication/03_structure_cible.sh
amine@OrdideAmine:~/td_migration$ bash niveau3_replication/03_structure_cible.sh
=== [NIVEAU 3] Creation de la structure sur la cible ===
DROP TABLE
Successfully copied 8.19kB to /home/amine/td_migration/backups/schema_only.sql
Successfully copied 8.19kB to pg_toulouse:/tmp/schema_only.sql
SET
SET
SET
SET
SET
set_config
-----
(1 row)
SET
SET
SET
SET
SET
SET
CREATE TABLE
CREATE SEQUENCE
ALTER SEQUENCE
CREATE TABLE
CREATE SEQUENCE
ALTER SEQUENCE
CREATE TABLE
CREATE SEQUENCE
ALTER SEQUENCE
CREATE TABLE
ALTER SEQUENCE
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
CREATE PUBLICATION
OK : Structure creee sur la cible
List of relations

```

Schema	Name	Type	Owner
public	formations	table	admin
public	progressions	table	admin
public	resultats_examens	table	admin
public	utilisateurs	table	admin

```

(4 rows)

nb_utilisateurs
-----
0
(1 row)
```

04_subscription.sh

```
#!/bin/bash

# niveau3_replication/04_subscription.sh

echo '=== [NIVEAU 3] Creation de la SUBSCRIPTION sur la cible ==='

# Recuperer l'IP du container source

SOURCE_IP=$(docker inspect pg_montpellier \

--format='{{range .NetworkSettings.Networks}}{{.IPAddress}}{{end}}')

echo "IP du container source : $SOURCE_IP"

# Creer la subscription sur la cible

docker exec pg_toulouse psql -U admin -d techcorp_db -c \
```

```

"CREATE SUBSCRIPTION sub_techcorp

CONNECTION 'host=$SOURCE_IP port=5432 dbname=techcorp_db user=admin password=admin123'

PUBLICATION pub_techcorp;"

echo 'OK : Subscription creee - replication en cours...'

sleep 5

# Verifier l'etat de la replication

docker exec pg_toulouse psql -U admin -d techcorp_db -c \

    "SELECT subname, subenabled FROM pg_subscription;"

# Verifier que les donnees sont arrivees

docker exec pg_toulouse psql -U admin -d techcorp_db -c \

    'SELECT COUNT(*) AS nb_utilisateurs FROM utilisateurs;'

# Etat detaille

docker exec pg_montpellier psql -U admin -d techcorp_db -c \

    "SELECT * FROM pg_stat_replication;"

```

```

amine@ordiDeAmine:~/td_migration$ nano ~/td_migration/niveau3_replication/04_subscription.sh
amine@ordiDeAmine:~/td_migration$ bash niveau3_replication/04_subscription.sh
=== [NIVEAU 3] Creation de la SUBSCRIPTION sur la cible ===
IP du container source : 172.19.0.2
NOTICE: created replication slot "sub_techcorp" on publisher
CREATE SUBSCRIPTION
OK : Subscription creee - replication en cours...
  subname | subenabled
-----+-----
 sub_techcorp | t
(1 row)

nb_utilisateurs
-----
8
(1 row)

pid | usesysid | username | application_name | client_addr | client_hostname | client_port |          backend_start          | backend_xmin | state | sent_lsn | write_l
sn | flush_lsn | replay_lsn | write_lag | flush_lag | replay_lag | sync_priority | sync_state |          reply_time          |
-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----
385 | 10 | admin | sub_techcorp | 172.19.0.3 | | | 54692 | 2026-05-19 09:43:05.171932+00 | | streaming | 0/19B7D40 | 0/19B7D
40 | 0/19B7D40 | 0/19B7D40 | | | | 0 | async | 2026-05-19 09:43:05.336016+00 |
(1 row)

```

05_test_replication.sh

```

#!/bin/bash

# niveau3_replication/05_test_replication.sh

echo '=== [NIVEAU 3] Test de la replication en temps reel ==='

echo '--- Avant insertion ---'

echo -n 'Source : '

docker exec pg_montpellier psql -U admin -d techcorp_db -t -c \

    'SELECT COUNT(*) FROM utilisateurs;'

echo -n 'Cible : '

docker exec pg_toulouse psql -U admin -d techcorp_db -t -c \

```

```
'SELECT COUNT(*) FROM utilisateurs;'
```

Insérer un nouvel utilisateur sur la SOURCE

```
docker exec pg_montpellier psql -U admin -d techcorp_db -c \
```

```
    "INSERT INTO utilisateurs (nom, email)
```

```
    VALUES ('Test Replication', 'test.replication@techcorp.fr');"

sleep 2

echo '--- Apres insertion (2 secondes plus tard) ---'

echo -n 'Source : '
```

```
docker exec pg_montpellier psql -U admin -d techcorp_db -t -c \
```

```
    'SELECT COUNT(*) FROM utilisateurs;'
```

```
echo -n 'Cible  : '
```

```
docker exec pg_toulouse psql -U admin -d techcorp_db -t -c \
```

```
    'SELECT COUNT(*) FROM utilisateurs;'
```

echo 'Si les deux comptages sont identiques, la replication fonctionne !'

Mesurer le lag de replication

```
echo '--- Lag de replication ---'
```

```
docker exec pg_montpellier psql -U admin -d techcorp_db -c \
```

```
    "SELECT application_name, state, sync_state,
```

```
    write_lag, flush_lag, replay_lag
```

```
    FROM pg_stat_replication;"
```

```
amine@OrdiDeAmine:~/td_migration$ nano ~/td_migration/niveau3_replication/05_test_replication.sh
amine@OrdiDeAmine:~/td_migration$ bash niveau3_replication/05_test_replication.sh
=== [NIVEAU 3] Test de la replication en temps reel ===
--- Avant insertion ---
Source :      8
Cible :      8
INSERT 0 1
--- Apres insertion (2 secondes plus tard) ---
Source :      9
Cible :      9
Si les deux comptages sont identiques, la replication fonctionne !
--- Lag de replication ---
application_name | state | sync_state | write_lag | flush_lag | replay_lag
-----
sub_techcorp    | streaming | async      | 00:00:00.000915 | 00:00:00.201705 | 00:00:00.000915
(1 row)
```

06_bascule_chaud.sh

```
#!/bin/bash

# niveau3_replication/06_bascule_chaud.sh
```

```

echo '=== [NIVEAU 3] Bascule a chaud ==='

echo "Debut : $(date)" | tee logs/coupure_n3.log

# 1. Verifier le lag de replication

echo '--- Lag de replication avant bascule ---'

docker exec pg_montpellier psql -U admin -d techcorp_db -c \

    "SELECT client_addr, state, sent_lsn, write_lsn, flush_lsn, replay_lsn

    FROM pg_stat_replication;"

# 2. Passer la source en lecture seule

docker exec pg_montpellier psql -U admin -d postgres -c \

    "ALTER DATABASE techcorp_db SET default_transaction_read_only = on;"

echo 'OK : Source en lecture seule'

# 3. Attendre que la replication soit complete

sleep 3

# 4. Supprimer la subscription sur la cible

docker exec pg_toulouse psql -U admin -d techcorp_db -c \

    "DROP SUBSCRIPTION sub_techcorp;"

echo 'OK : Subscription supprimee'

# 5. Activer la cible en lecture/ecriture

docker exec pg_toulouse psql -U admin -d techcorp_db -c \

    "ALTER DATABASE techcorp_db SET default_transaction_read_only = off;"

echo 'OK : Cible activee'

echo "Fin de la bascule : $(date)" | tee -a logs/coupure_n3.log

echo 'OK : Migration avec replication terminee'

# Comparaison des durees

echo ''

echo '=== Comparaison des coupures ==='

echo "N1 Big Bang      : ~4 minutes"

echo "N2 Progressive    : ~2 secondes"

echo "N3 Replication     : quelques secondes"

```

```

niveaureplication@td_migration$ nano ~/td_migration/niveau3_replication/06_bascule_chaud.sh
niveaureplication@td_migration$ bash niveau3_replication/06_bascule_chaud.sh
=== [NIVEAU 3] Bascule a chaud ===
Debut : Tue May 19 11:40:39 CEST 2026
--- Lag de replication avant bascule ---
client_addr | state | sent_lsn | write_lsn | flush_lsn | replay_lsn
-----
172.19.0.3 | streaming | 0/198B3F8 | 0/198B3F8 | 0/198B3F8 | 0/198B3F8
(1 row)

ALTER DATABASE
OK : Source en lecture seule
NOTICE: dropped replication slot "sub_techcorp" on publisher
DROP SUBSCRIPTION
OK : Subscription supprimee
ALTER DATABASE
OK : Cible activee
Fin de la bascule : Tue May 19 11:44:43 CEST 2026
OK : Migration avec replication terminee

=== Comparaison des coupures ===
N1 Big Bang      : ~4 minutes
N2 Progressive    : ~2 secondes
N3 Replication     : quelques secondes

```


07_nettoyage.sh

```
#!/bin/bash

# niveau3_replication/07_nettoyage.sh

echo '=== [NIVEAU 3] Nettoyage post-migration ==='

# Supprimer la publication sur la source
docker exec pg_montpellier psql -U admin -d techcorp_db -c \

    "DROP PUBLICATION IF EXISTS pub_techcorp;"

echo 'OK : Publication supprimée sur la source'

# Verification finale
echo '--- Etat final de la cible ---'

docker exec pg_toulouse psql -U admin -d techcorp_db -c \

    "SELECT 'utilisateurs', COUNT(*) FROM utilisateurs

    UNION ALL SELECT 'formations', COUNT(*) FROM formations

    UNION ALL SELECT 'progressions', COUNT(*) FROM progressions

    UNION ALL SELECT 'resultats_examens', COUNT(*) FROM resultats_examens;"
```

```
amine@OrdiDeAmine:~/td_migration$ nano ~/td_migration/niveau3_replication/07_nettoyage.sh
amine@OrdiDeAmine:~/td_migration$ bash niveau3_replication/07_nettoyage.sh
=== [NIVEAU 3] Nettoyage post-migration ===
ERROR:  cannot execute DROP PUBLICATION in a read-only transaction
OK : Publication supprimée sur la source
--- Etat final de la cible ---
   ?column?   | count
-----+-----
 utilisateurs |     9
 formations   |     4
 progressions |     6
 resultats_examens |     4
(4 rows)
```

Résultat final — Bascule à chaud

Début bascule : 11:44:39 | Fin bascule : 11:44:43 | Durée : ~4 secondes

Lag de réplication mesuré : < 1ms (write_lag : 00:00:00.000915) — Zéro perte de données.

Questions d'analyse – Niveau 3

Q1 : Quelle est la durée de coupure avec la réplication logique ? Comparez.

N1 Big Bang : ~4 minutes | N2 Progressive : ~2 secondes | N3 Réplication : ~4 secondes.
La réplication offre une coupure quasi nulle — le délai est uniquement lié à l'attente du dernier flush WAL, pas au volume de données.

Q2 : Quelles sont les limites de la réplication logique PostgreSQL ?

- Ne réplique pas le schéma automatiquement (DDL).
- Les séquences ne sont pas synchronisées — risque de collision d'ID après bascule.
- Nécessite wal_level = logical — légère surcharge sur le serveur source.
- Pas de réplication des Large Objects.
- La cible ne peut pas recevoir d'écritures pendant la réplication.

Q3 : Pourquoi doit-on créer le schéma manuellement avant la subscription ?

La réplication logique PostgreSQL ne réplique que les données (DML : INSERT, UPDATE, DELETE), pas la structure (DDL). Elle a besoin que les tables existent déjà sur la cible. Sans le schéma préalable, la subscription échoue avec une erreur 'relation does not exist'.

Q4 : Que se passe-t-il si on insère une ligne sur la CIBLE pendant que la réplication est active ?

L'insertion réussit techniquement mais crée un risque de conflit de données : si la même clé primaire est générée sur source et cible, la réplication échoue sur cette ligne avec une erreur de violation de contrainte unique. C'est pourquoi la cible doit rester en lecture seule pendant la réplication.

Q5 : Dans quel cas préférerait-on la réplication logique à la migration progressive ?

- Quand la base est très volumineuse et que même 2 secondes de coupure sont difficiles à planifier.
- Pour des systèmes à haute disponibilité où la continuité de service est critique.
- Quand on veut valider la cible en production avant de basculer (la cible reçoit les données en temps réel).
- Pour des migrations progressives avec cohabitation source/cible pendant plusieurs jours.

Q6 : Quels outils professionnels permettent une réplication encore plus avancée ?

- pglogical : extension PostgreSQL pour la réplication logique avancée avec gestion des conflits.
- Debezium : CDC (Change Data Capture) basé sur Kafka, multi-bases de données.
- AWS DMS (Database Migration Service) : service managé pour migrations cloud.
- Patroni : haute disponibilité PostgreSQL avec failover automatique.
- BDR (Bi-Directional Replication) : réplication multi-maîtres.

6. Synthèse comparative des 3 méthodes

Critère	Big Bang (N1)	Progressive (N2)	Réplication (N3)
Durée de coupure mesurée	~4 minutes	~2 secondes	~4 secondes
Complexité de mise en œuvre	Faible	Moyenne	Élevée
Risque de perte de données	Élevé	Moyen	Faible
Rollback possible ?	Oui	Oui	Oui
Adapté à 2h de coupure ?	Oui	Oui	Oui
Lag de réplication	N/A	N/A	< 1ms
Outils utilisés	pg_dump/rsync	pg_dump + rsync	PUBLICATION/SUBSCRIPTION
Recommandé pour ce projet ?	Non	Oui (compromis)	Oui (meilleure option)

7. Conclusion et recommandation

Recommandation pour TechCorp Formation : Réplication Logique PostgreSQL (Niveau 3)

Parmi les trois méthodes testées, la réplication logique PostgreSQL (Niveau 3) est la solution la mieux adaptée à la migration de TechCorp Formation de Montpellier vers Toulouse.

Elle offre une coupure quasi nulle (~4 secondes mesurées), un risque de perte de données minimal grâce à la synchronisation continue en temps réel, et permet de valider la cible en conditions réelles avant la bascule définitive. Le lag de réplication mesuré est inférieur à 1ms, ce qui garantit une cohérence quasi parfaite entre source et cible au moment du basculement.

La migration progressive (N2) reste une excellente alternative si l'infrastructure ne supporte pas la réplication logique, ou si la complexité de mise en œuvre est un frein. Ses 2 secondes de coupure sont largement dans la fenêtre de 2 heures autorisée.

La migration Big Bang (N1) est à éviter en production : bien qu'elle ait fonctionné en 4 minutes sur cette petite base de test, elle serait bien plus longue sur une base réelle, et tout incident pendant la restauration bloquerait le service sans possibilité de reprise rapide.